Reference: ML System Design by ByteByteGo.

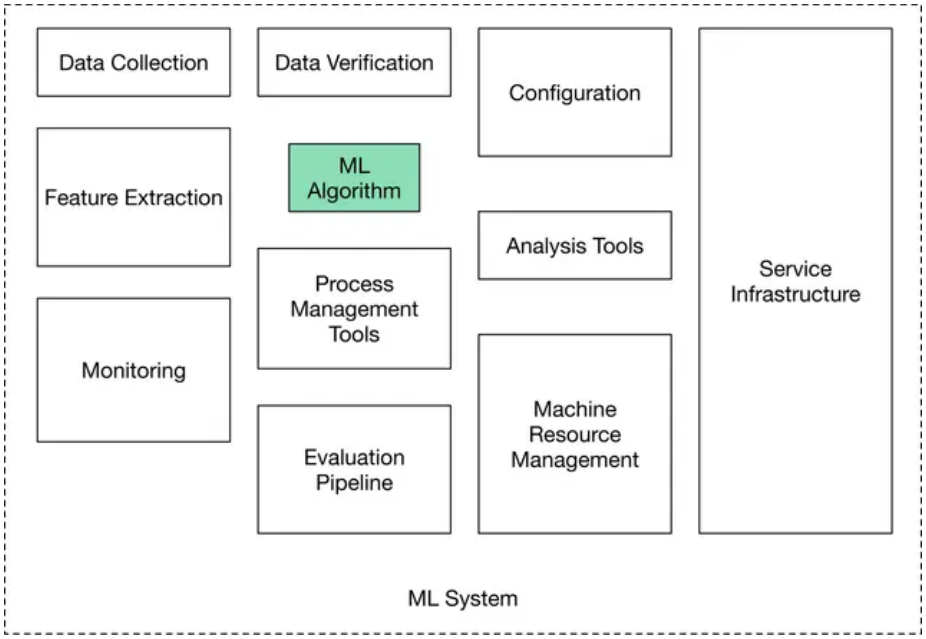

Figure 1: Components of a production ready ML system.

Introduction and Overview

ML system design interviews expect us to answer open-ended questions. There is no single correct answer. We are evaluated on our thought process, our in-depth understanding of various ML topics, our ability to design an end-to-end system, and our design choices based on the trade-offs of various options.

Unstructured answers make the flow difficult to follow and derail open-ended discussions. It is important to follow a framework:

Figure 2: ML system design steps.

- Clarify the objective to help shape primary requirements. Transform the objective/ requirements into a tangible ML task.
- Data preparation is often overlooked. Don't be an idiot. Think through what will be fed to the model, both in terms of `X` and `Y`.

- Model development and evaluation must be an continuously iterative process. To `MeasureEverything` is important; ideally evaluation (and non-ML baselines) must be decided in the early stages of model development.
- Continuous monitoring after serving is essential to ensure that the objectives are being met and that the data/ concept does not drift.

Clarify Requirements

Ask clarifying questions; asking questions exhibits attention to detail and intention. Attempt to understand the exact requirements and what the user (the interviewer) cares about:

- **Business objective**: Increase revenue? Increase #users? Increase engagement per user?
- **Features the system needs to support**: Move towards tangible features. Clarify if the features are essential or good-to-haves. Discuss feasibility while keeping a realistic ML system in mind. For example, a recommendation system might allow users to "like" or "dislike" recommendations, as those interactions could be used to label training data.
- **Data**: Data sources? How large is a datum and the dataset? Is it structured or unstructured? Do we have labels and is there a possibility for soft labels?
- **Constraints**: How much computing power is available (during inference; during learning we assume ∞ resources)? Is it a cloud-based system, or should the system work on a device? Is the model expected to improve automatically over time?
- **Scale of the system**: How many users do we have? How many items, such as videos, are we dealing with? What's the rate of growth of these metrics?
- **Performance**: How fast must inference be? Is a real-time solution expected? Does accuracy have more priority or latency?
- **FairML**: Do Fairness, Accountability, Transparency, and Explainability matter?

Frame the Problem as an ML Task

Effective problem framing is essential for translating a business objective into a clear ML task. This involves `(1)` ensuring that ML is necessary (though an ML system design interview probably requires an ML system...), `(2)` defining a precise ML objective that models can address (e.g., maximizing click-through

rate instead of merely "increasing sales"), (3) specifying the system's inputs and outputs as a blackbox — especially when multiple models are involved, and (4) choosing the appropriate ML paradigm (e.g., supervised learning for most cases, with further distinctions between binary/ multiclass classification or regression), ensuring the task aligns with the nature of available data and the intended outcome.

Define the ML Objective Possible business objective: increase sales by 20%. But, we cannot optimize over sales directly. For an ML system to solve a task, we must translate the business objective into a well-defined ML objective (say, to improve recommendations so users buy more items). We could also increase sales by increasing the #users by 20%, but that might require moving away from improving recommendations towards, say, a better ad-serving system. Increasing sales can only be measured on the field and that is not a good ML objective for learning and evaluating the ML system.

A *good ML objective* is one that ML models can solve and be evaluated upon.

Specify the System's I/O As a black-box what are X and Y? In some cases, the system may be complex and we may need to specify the I/O of each component ML model. There may be more than one way to specify the I/O and these decisions matter for how we make downstream decisions.

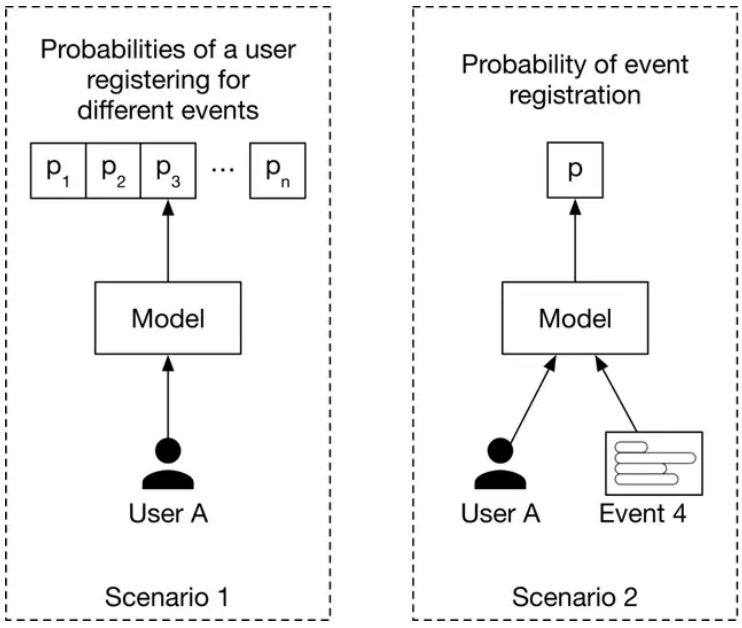

Figure 3: Different ways to specify the model's I/O.

Choose the Right ML Paradigm Generally, the solution will involve supervised learning. However, do not assume this. Also, its important to drill down into whether its classification (binary vs. multi-label/ nominal vs. ordinal) or regression. Suprisingly, weakly supervised learning is not discussed at all.

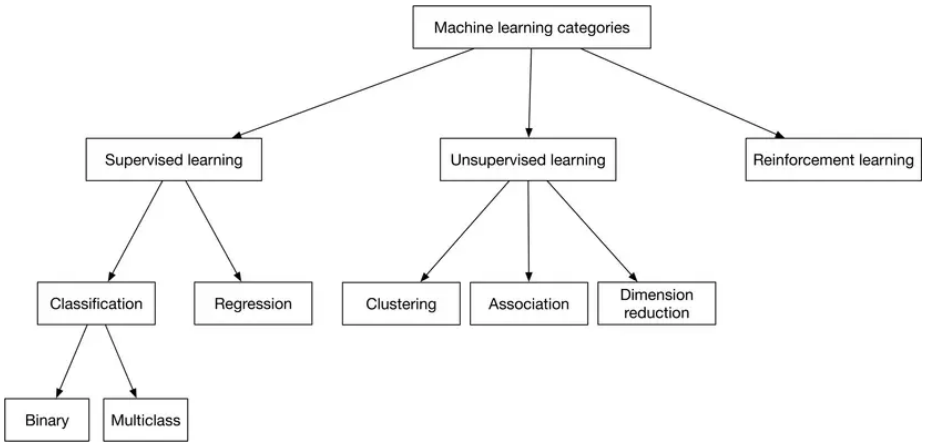

Figure 4: Common ML categories. Significant miss: weakly supervised learning.

Data Preparation

Data with predictive power is essential for ML. Two essential processes: Data Engineering and Feature Engineering.

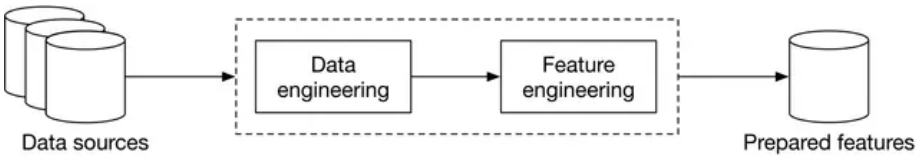

Figure 5: Data preparation process.

→ Data Engineering

Design and build pipelines for collecting/ storing/ retrieving/ processing data.

Data Sources/ Data Storage

Data Sources: Understanding the data source provides context for label hygiene/ noise and general reliability. Metadata tags alongwith data are often a valuable source of information.

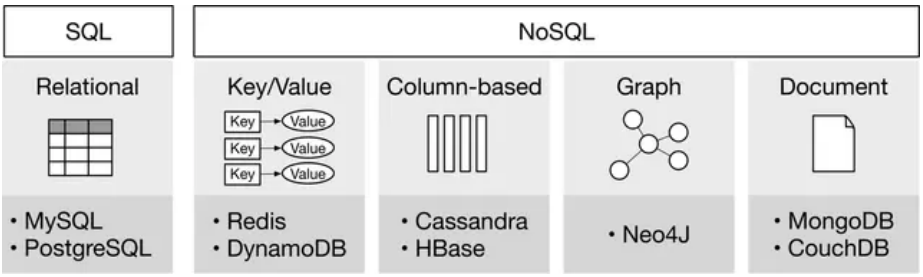

Figure 6: Types of databases.

Data Storage: Repository for peristently storing + managing collections of data.

Extract, Transform, and Load (ETL) consists of three phases:

1. Extract: Extracts data from heterogenous data sources.
2. Transform: Data is cleansed, mapped, and transformed to meet operational needs.
3. Load: The transformed data is loaded into the target destination.

Data Types: Structured and unstructured data, with a number of different sub-types exist.

- Numerical: Discrete numbers.
- Categorical: Names/ labels.
  - Nominal: No numeric relationship.
  - Ordinal: Ordering exists.

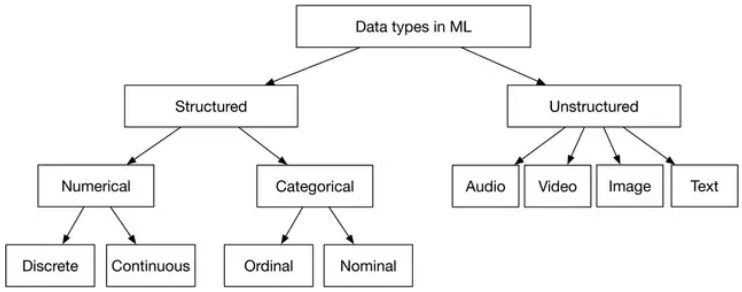

Figure 7: Types of data.

Table 1: Summary of structured and unstructured data

	Structured	Unstructured
Characteristics	- Predefined schema - Easy to search	- No schema - Difficult to search
Resides in	- Relational databases - Many NoSQL databases can store structured data - Data warehouses	- NoSQL databases - Data lakes
Examples	- Dates - Phone numbers - Credit card numbers - Addresses - Names	- Text files - Audio files - Images - Videos

→ **Feature Engineering**

Feature engineering process requires subject matter expertise and is highly dependent upon the task at hand. Two processes:

- Using domain knowledge to select and extract predictive features from raw data.
- Transforming predictive features into a format usable by the model.

Three important operations: * Handle missing data: delete or impute. * (Skewed) feature scaling. ML models (like SVMs) may struggle to learn when the features are in different ranges. Some models may struggle when a feature has a skewed distribution and the model expects each feature to be either linearly or normally distributed. Techniques: min-max-normalization, Z-score-standardization, log scaling, exponentiation, and discretization. * Encode categorical variables: integer encoding, one-hot encoding (fair for a small number of possible values), and embedding learning (overcomes one-hot encoding and provides a continuous manifold for representing data).

Its important to generally look out for biases in the data. This should happen before Model Development begins.

Model Development

`NOTE`: It is an oversight on the author's part to not discuss (atleast) preliminary evaluation, say as a black-box for what the model is expected to be capable of doing well. Evaluation must be independent of Model Development and ideally,

should precede model development. This sidesteps the scenario where we choose a model that is not well suited to all our business needs, and then overfit our evaluation to what it is capable of doing or worse, already does well.

→ Model Selection

Select an appropriate ML model and train it to solve the task at hand:

- Establish a simple baseline. Start with setting up a non-ML baseline (random chance/ median/ mode) which will help us set the lower bound for what a data-driven model is expected to outperform.
- Experiment with simple models. After we have a baseline, explore ML algorithms that are quick to train (but might require more feature engineering). Then, try to get as far as possible without neural nets. This also allows us to start getting a sense of the data and what features are useful at the beginning of the dev cycle where we might not have a lot of data.
- Switch to more complex models; embrace end-to-end learning.
- Use an ensemble of models to improve performance.

Considerations when choosing an ML model:

- The amount of data the model needs.
- Hyperparameter space complexity.
- Possibility of continual learning.
- Compute requirements (especially during inference).

→ Model Training

Steps:

- Constructing the dataset/ engineering features.
- Selecting supervisory signals (crafting loss functions).
- Training from scratch vs. fine-tuning.
- Distributed training (important in big-tech).

Constructing the dataset

Steps:

- Collect raw data. Discussed above.
- Identify features and labels. Features depend on the task. Labels can be *labeled* in many ways

  - Hand labeling: Reliable quality but slow and expensive. Can be reserved for either finetuning or for evaluating.
  - Label mining: User activity can be transformed into supervision.

- Sampling
  - Convenience Sampling: Select participants/ datapoints who are readily accessible. Quick, but may lead to bias.
  - Snowball Sampling: Existing datapoints recruit new datapoints. May introduce biases as the sample may not be representative of the entire population.
  - Stratified Sampling: Divide population into distinct subgroups, or strata, that share similar characteristics. Random samples are then drawn from each stratum, ensuring representation across key segments of the population. Enhances the accuracy and reliability of results, especially when analyzing diverse populations.
  - Reservoir Sampling: A randomized algorithm is used for selecting a random sample of `k` items from a population of unknown size `n`. Useful when dealing with data streams or when the total population size is not known in advance.
  - Importance Sampling: A statistical technique used to estimate properties of a particular distribution while only having samples generated from a different distribution. Employed when direct sampling is challenging, allowing for the evaluation of expectations under one distribution by using samples from another.

- Address class imbalance while making sure that train-tune-test splits have the same ratios.
  - Undersampling majority class.
  - Oversampling minority class.
  - Address during optimization by re-weighting the loss. Popular choices are $\frac{1}{n}$ and $\frac{1}{n \log n}$ and the focal loss ($(1 - p_k)^\gamma$ where usually $\gamma = 2$).

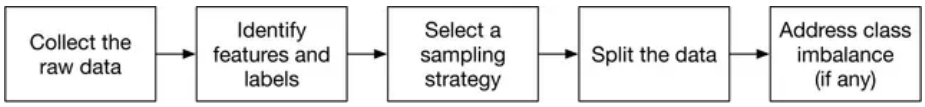

Figure 8: Dataset construction steps.

Choose supervisory signals/ loss functions

The loss function allows the optimization to update the model's parameters during learning. With end-to-end learning crafting the right loss becomes extremely important. Domain knowledge and business needs must inform the design of loss functions.

Often, figuring this out is as fun (if not more) as choosing the right neural network and throwing transformers at the problem. It does require reliable engineers who know what they are doing.

Distributed Training

Two methods: Data parallelism and model parallelism. Data parallelism involves replicating the entire model across multiple devices, with each device processing a different subset of the data simultaneously; gradients from each device are then aggregated to update the model parameters collectively. Model parallelism partitions a single model across multiple devices, where each device is responsible for computing a portion of the model's operations, making it suitable for models too large to fit into a single device's memory.

Generally, unless we're training LLMs, data parallelism (in the form of multi-GPU training) is sufficient.

Evaluation

Two types:

- Offline Eval: During learning and tuning. Generally, the MLE will decide these.
- Online Eval: During monitoring after deployment. Generally, business needs/ objectives will inform these.

This is too simplistic. There's not much to learn from what is discussed by the authors. Instead, read Model Evaluation, Model Selection, and Algorithm Selection in Machine Learning by Raschka.

Deployment and Serving

Aspects:

- Cloud vs. on-device deployment
- Model compression
- Testing in production
- Prediction pipeline

→ Cloud vs. on-device deployment

- Cloud is simpler to deploy to compared to on-device deployment.
- Inference is faster on the cloud and has fewer hardware constraints.
- Network latency makes on-device deployment more appropriate.
- On-device deployment ensures user privacy.

→ Model compression

Make models smaller to reduce inference latency and compute requirement. Three techniques:

- Knowledge distillation: Train a small (student) model to mimic a larger (teacher) model.
- Pruning: Zero-ing out the least useful parameters. Leads to sparser models.
- Quantization: Use fewer bits to represent the parameters, leading to quantization of network parameters.

→ Testing in production

The ultimate way to ensure real-world performance is to test the model with real traffic. Commonly used techniques include shadow deployment, A/B testing, canary release, interleaving experiments, and bandits, etc.

A/B Testing

Deploy the new model in parallel with the existing model. Two important factors to consider:

- Traffic routed to each model has to be random.
- A/B tests should be run on a sufficient number of users/ data points for the results to be legitimate. `5%/95%` is a reasonable split.

Interleaving Experiments

- Merge outputs from two or more models into a single, combined result. User interactions with interleaved output are then analyzed to infer preferences between the models.
- Offers higher sensitivity compared to traditional A/B testing, allowing for more rapid and nuanced comparisons.
- Particularly useful in scenarios like search engine result rankings or recommendation systems, where subtle differences between models can significantly impact user experience.
- Best suited for direct, pairwise comparisons where the goal is to detect subtle performance differences between models. Provide quick insights, but are typically limited to comparing two models at a time.

Multi-Armed Bandits

- Address the exploration-exploitation dilemma by dynamically allocating traffic to different models based on their performance.

- Useful to balance the introduction of new models (exploration) with the utilization of established, high-performing models (exploitation).
- Enables continuous monitoring and adjustment, facilitating real-time responses to model performance variations and reducing the need for manual intervention
- Ideal for scenarios involving multiple models or strategies, where the objective is to maximize overall performance over time. Effective in dynamically changing environments, allowing for continuous adaptation to new data or user behaviors.

Shadow Deployment

- Deploy the new model in parallel with the existing model. Incoming requests are routed to both models, but only the existing model's prediction is served to the user.
- This minimize the risk of unreliable predictions but, is a costly approach that doubles the compute requirements.

→ Prediction pipeline

Two types: batch predictions and online predictions.

- Batch predictions involve generating predictions for a large set of data periodically. Typically employed when immediate results are not critical, and predictions can be made in bulk.
- Batch Predictions are used in the following scenarios:
  - Non-Time-Sensitive Tasks: Predictions are not required in real-time, such as generating nightly reports or updating recommendation systems periodically.
  - Large-Scale Data Processing: Larger volumes can be processed more efficiently in bulk.
  - Resource Optimization: Compute on intensive tasks can be optimized by scheduling during off-peak hours, thereby reducing costs and balancing load.
- Advantages of Batch Predictions:
  - Efficiency: Processing large datasets collectively can be more resource-efficient than handling numerous individual requests.
  - Simpler Infrastructure: Batch processing systems are generally less complex, as they don't require the low-latency infrastructure needed for real-time predictions.
- Disadvantages of Batch Predictions:
  - Latency: Not suitable for applications requiring immediate predictions.

  - Data Staleness: Predictions may become outdated between processing intervals, which is problematic in dynamic environments.

- Online (or real-time) pipelines generate predictions on-demand, processing individual data points as they arrive. Essential for applications where timely responses are critical, such as fraud detection or personalized user experiences.
- When Online Predictions Are Preferred:
  - Immediate Response Required: Applications like real-time bidding in advertising or instant recommendation systems.
  - Rapidly Changing Data: Environments where data evolves quickly, necessitating up-to-date predictions.
  - All interactive systems are online predictors.

Trade-offs: * Complexity vs. Latency: Online systems require more complex infrastructure to handle low-latency demands, whereas batch systems are simpler but introduce latency due to their scheduled nature. * Resource Allocation: Batch processing can be scheduled during low-demand periods to optimize resource usage, while online systems must maintain resources to handle peak loads at any time.

The choice between batch and online prediction pipelines depends on the specific requirements of the application, including the need for immediacy, resource availability, and the nature of the data being processed.

→ Monitoring and Infrastructure

NOTE: Like the evaluation section, this section lacks depth. Online evaluation and continual learning are critical for real-world ML systems.

Why systems fail

- Data distribution drift: The data a model encounters in production differs from that it encountered during training.
  - We must continuously model the online data distribution and compare it to the training data distribution.
  - Training on large datasets to learn a comprehensive distribution helps.
  - Regularly retraining the model using labeled data from the new distribution helps more.
  - Personalizing models to subsets of users based on demographics and evolving preferences helps *even* more.
- Concept drift: A change in the underlying data distribution or the target concept over time in real-world ML systems, i.e., the relationship between input features and the target variable evolves, which can negatively impact model performance if not addressed.

  - Example: financial market prediction. A model that forecasts stock prices based on historical trends might gradually lose accuracy as market conditions, investor behavior, and regulatory environments shift. Hence, the relationship between market indicators and price movements changes, representing concept drift.
  - Concept drift is more tricky to model directly compared to data drift and requires the model to be updated or retrained to capture new patterns. It may even require major changes to the design of the underlying ML model and feature engineering techniques.

What to monitor

Detect failures and identify shifts in the ML system. Two categories:

- Operational metrics: Ensure the system is up and running: average serving time, throughput, number of prediction requests, CPU/GPU utilization, etc.
- ML metrics:
  - Monitoring inputs/outputs. Avoid garbage-in-garbage-out. Monitoring the model's input and outputs is vital.
  - Drifts. Inputs to the system and the model's outputs are monitored to detect changes in their underlying distribution.
- Business metrics. Real-world systems must make sure that business needs are met. Neither operational nor ML metrics are useful when business need fails.

Takeaway

No engineer can be an expert in every aspect of the ML lifecycle. Some engineers specialize in deployment and production, while others specialize in model development. Data science generally requires more data engineering, while applied ML focuses more on model development and productionization. A (successful) candidate should seek to drive the conversation, while being ready to go with the interviewer's flow. This is why frameworks like the one discussed in this document help.